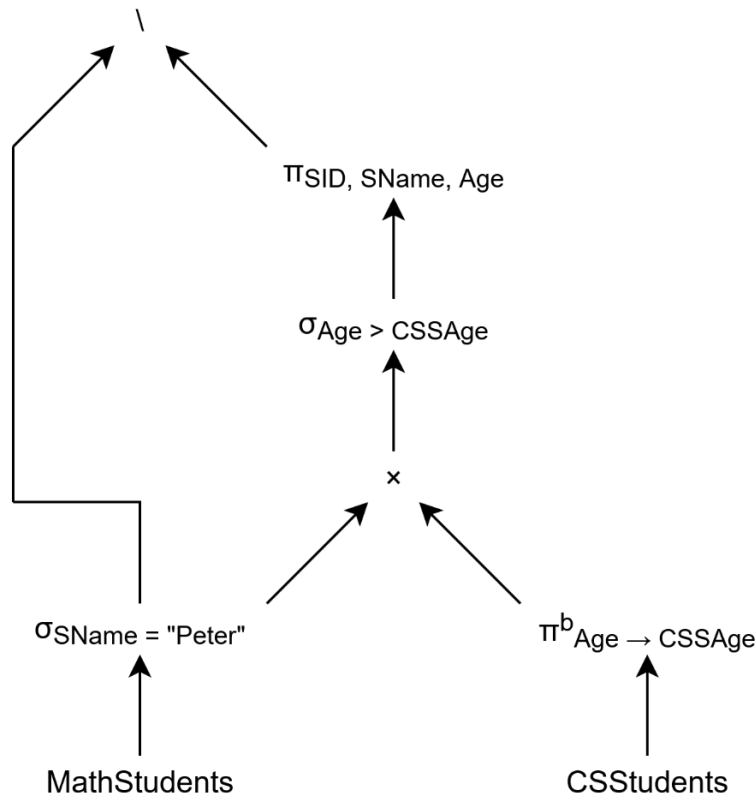


Question 1



The tree above is the one I came up with. Note that the order for the set difference operator matters, and that the whole tree is equivalent to: $\sigma_{SName = "Peter"}(MathStudents) \setminus$

$$\pi_{SID, SName, Age} \left(\sigma_{Age > CSSAge} \left(\sigma_{SName = "Peter"}(MathStudents) \times \pi_{Age \rightarrow CSSAge}^b(CSSStudents) \right) \right)$$

Question 2

Part a.

Relation/Statistics	Size(R)	Tups(R)	TupSize(R)	Vals(R.X)
MathStudents	$3 * 50 = 150$	3	$10 + 30 + 10 = 50$	$X = SID: 3$ $X = SName: 2$ $X = Age: 2$
CSSStudents	$3 * 50 = 150$	3	$10 + 30 + 10 = 50$	$X = SID: 2$ $X = SName: 2$ $X = Age: 3$
Takes	$3 * 40 = 120$	3	$10 + 30 = 40$	$X = SID: 2$ $X = Course: 2$

Above is the table of statistics requested of each of the relations for the cost function.

P.S. all the formulas and methods used in both part a and part b are taken from lecture 13.

Part b.

The cost function is the sum of the sizes after each operation of the whole expression. So the whole process becomes: $TotalCost = Size(\sigma_{SName="Julia"}(MathStudents)) + Size(\sigma_{SName="Julia"}(MathStudents) \cap CSStudents) + Size((\sigma_{SName="Julia"}(MathStudents) \cap CSStudents) \bowtie Takes)$

Notice the propagation, for easier visualization purposes we will compute each one separately and use the previous steps in the next ones, then sum it all up in the end.

1. $Size(\sigma_{SName="Julia"}(MathStudents)) =$
 $= Tups(\sigma_{SName="Julia"}(MathStudents)) * TupSize(MathStudents) =$
 $= \frac{Tups(MathStudents)}{Vals(MathStudents.SName)} * TupSize(MathStudents) = \frac{3}{2} * 50 = 1.5 * 50 = 75$
2. $Size(\sigma_{SName="Julia"}(MathStudents) \cap CSStudents) =$
 $= Tups(\sigma_{SName="Julia"}(MathStudents) \cap CSStudents) * TupSize(CSStudents) =$
 $= \frac{\min(Tups(\sigma_{SName="Julia"}(MathStudents)), Tups(CSStudents))}{2} * TupSize(CSStudents) =$
 $= \frac{\min(1.5, 3)}{2} * 50 = \frac{1.5}{2} * 50 = 0.75 * 50 = 37.5$, note that $TupSize(CSStudents) =$
 $TupSize(\sigma_{SName="Julia"}(MathStudents))$ must hold due to $\cap$ operator.
3. For this one we need $Vals((\sigma_{SName="Julia"}(MathStudents) \cap CSStudents).SID)$
 which we did not keep track of, so let's catch up and compute it now:
 $Vals((\sigma_{SName="Julia"}(MathStudents) \cap CSStudents).SID) =$
 $= \frac{\min(Vals(\sigma_{SName="Julia"}(MathStudents).SID), Vals(CSStudents.SID))}{2} =$
 $= \frac{\min\left(\frac{Vals(MathStudents.SID) * Tups(\sigma_{SName="Julia"}(MathStudents))}{Tups(MathStudents)}, Vals(CSStudents.SID)\right)}{2} =$
 $= \frac{\min(\frac{3*1.5}{3}, 2)}{2} = \frac{\min(1.5, 2)}{2} = \frac{1.5}{2} = 0.75$. Now we can continue and compute the size.
 $Size((\sigma_{SName="Julia"}(MathStudents) \cap CSStudents) \bowtie Takes) =$
 $= Tups((\sigma_{SName="Julia"}(MathStudents) \cap CSStudents) \bowtie Takes) *$
 $* TupSize((\sigma_{SName="Julia"}(MathStudents) \cap CSStudents) \bowtie Takes) =$

$$\begin{aligned}
&= \frac{Tups(\sigma_{SName="Julia"}(MathStudents) \cap CSStudents) * Tups(Takes)}{\max(Vals((\sigma_{SName="Julia"}(MathStudents) \cap CSStudents).SID), Vals(Takes.SID))} * \\
&* (TupSize((\sigma_{SName="Julia"}(MathStudents) \cap CSStudents)) + TupSize(Takes) - Size(SID)) = \\
&= \frac{0.75 * 3}{\max(0.75, 2)} * (TupSize(CSStudents) + TupSize(Takes) - Size(SID)) = \\
&= \frac{2.25}{2} * (50 + 40 - 10) = 90. \text{ Note that } Size(SID) = \text{Size of value in } B = 10.
\end{aligned}$$

So, the $TotalCost = 75 + 37.5 + 90 = \underline{202.5}$.

Question 3

Part a.

$$BlockSize = 8000B, \quad TupSize = 300B, \quad SearchKeySize = 20B, \quad PointerSize = 4B$$

The blocks are continuous which means that we don't need to save information on where the next block corresponding to the same table is, if there is any (unlike in a linked version where we do have that). Since it is unspanned as well, if we can't fit an entire tuple into a block, we try out the next one, and so on... So basically, we look at how many tuples a block can fit (unlike in spanned version where we would need to consider individual attributes).

An index consists of blocks of entries where each entry is made up of a *SearchKey* and a *Pointer* which means that its size is: $EntrySize = SearchKeySize + PointerSize = 20B + 4B = 24B$. A single block of entries contains $EntriesPerBlock = \left\lfloor \frac{BlockSize}{IndexSize} \right\rfloor = \left\lfloor \frac{8000B}{24B} \right\rfloor = 333$. Note that the floor function is there because the blocks are unspanned.

Because we compare by the least amount of block access done in worst-case scenarios, first we need to know the number of reads that the one-level dense index, which we will call *A* for short, needs, and that is $Reads_A = \left\lceil \frac{Tuples}{EntriesPerBlockLevels} \right\rceil + Levels = \left\lceil \frac{Tuples}{333^1} \right\rceil + 1$ where *Tuples* represents the number of tuples that our relation table has. Also, we add *Levels* which is 1 because after reading the first level, which is a dense index block, we need to access the block where the tuple data is actually stored. However, that detail only holds for when $Tuples > 0$, because otherwise there is no index. Next, the number of reads that the two-level index, with second-level sparse and first-level dense, which we will call *B*, has $Reads_B = \left\lceil \frac{Tuples}{EntriesPerBlockLevels} \right\rceil + Levels = \left\lceil \frac{Tuples}{333^2} \right\rceil + 2$ for when $Tuples > 0$. The reason behind it is identical to that of +1 in $Reads_A$, it's just a tree of greater depth, namely: 2.

Now, to find out when *B* is faster than *A* we calculate at what number of *Tuples* does $Reads_A > Reads_B$ hold, which we assume it will after *Tuples* crosses a certain threshold.

$$\begin{aligned}
& 1) Reads_A > Reads_B, \quad 2) \left\lceil \frac{Tuples}{333^1} \right\rceil + 1 > \left\lceil \frac{Tuples}{333^2} \right\rceil + 2, \quad 3) \left\lceil \frac{Tuples}{333} \right\rceil + 1 - \left\lceil \frac{Tuples}{333^2} \right\rceil > 2, \\
& 4) \left\lceil \frac{Tuples}{333} \right\rceil > 2, \quad 5) Tuples > 2 \cdot 333, \quad 6) Tuples > 666
\end{aligned}$$

So, the threshold for when the two-level index with 1st level dense and 2nd level sparse is more efficient than the one-leveled dense index is when the number of *Tuples* is above 666 which is basically 667 or more.

Part b.

Here we use similar formulas as in part a, so I won't cover these here. Also, note that the constant *EntriesPerBlock* has not changed, i.e. its value is identical to the one in part a. Now, Let's call the one-level sparse index *A*, and the two-level sparse index *B*. Since the first level in both of these is sparse, we must use $\frac{Tuples}{TuplesPerBlock}$ instead of just *Tuples* where $TuplesPerBlock = \left\lfloor \frac{BlockSize}{TupSize} \right\rfloor = \left\lfloor \frac{8000B}{300B} \right\rfloor = 26$. That is because unlike in a first-level dense index where each entry points to each tuple, i.e. a block and an offset within that block, each entry in a first-level sparse index points to the first tuple within a block which can hold multiple *Tuples* at once. Thus, the slightly modified formula for the number of reads is:

$$Reads_A = \left\lceil \frac{\frac{Tuples}{TuplesPerBlock}}{EntriesPerBlock^{Levels}} \right\rceil + Levels = \left\lceil \frac{\frac{Tuples}{26}}{333^1} \right\rceil + 1 \text{ for } A \text{ where } Tuples > 0, \text{ and}$$

$$Reads_B = \left\lceil \frac{\frac{Tuples}{TuplesPerBlock}}{EntriesPerBlock^{Levels}} \right\rceil + Levels = \left\lceil \frac{\frac{Tuples}{26}}{333^2} \right\rceil + 2 \text{ for } B \text{ where } Tuples > 0.$$

Now, we find out when $Reads_A > Reads_B$, i.e. when *A* is less efficient than *B*:

$$\begin{aligned}
& 1) Reads_A > Reads_B, \quad 2) \left\lceil \frac{\frac{Tuples}{26}}{333^1} \right\rceil + 1 > \left\lceil \frac{\frac{Tuples}{26}}{333^2} \right\rceil + 2, \quad 3) \left\lceil \frac{\frac{Tuples}{26}}{333} \right\rceil + 1 - \left\lceil \frac{\frac{Tuples}{26}}{333^2} \right\rceil > 2, \\
& 4) \left\lceil \frac{\frac{Tuples}{26}}{333} \right\rceil > 2, \quad 5) Tuples > 2 \cdot 333 \cdot 26, \quad 6) Tuples > 17316
\end{aligned}$$

So, the threshold for when the two-level index with both levels sparse is more efficient than the one-leveled sparse index is when the number of *Tuples* is above 17316 which is basically 17317 or more.